Defect Prediction and What-If Analysis

Flavio Di Palma - 0366635

January 6, 2026

Abstract

This report presents a comprehensive analysis of defect prediction in software systems using ML classifiers. I have conducted a what-if analysis on two Apache projects (BOOKKEEPER and SYNCOPE) to evaluate the potential impact of refactoring actionable features on bug prevention. My methodology involved dataset generation from Git and JIRA histories, feature extraction at method-level granularity, classifier training and evaluation, and simulation of refactoring scenarios. The results demonstrate that reducing actionable features can lead to a reduction in predicted defects, with drop rates of 18.16% for BOOKKEEPER and 21.42% for SYNCOPE. The codebase maintains zero code smells on SonarCloud, ensuring high code quality

Github repo: https://github.com/Flaasks/ISW2-DefectPredictionProject

1 Introduction

Software defect prediction is a crucial aspect of modern software engineering, enabling development teams to identify and address potential issues before they manifest in production environments. This project implements a comprehensive pipeline for defect prediction and what-if analysis, answering the fundamental question: *How many buggy methods could be prevented by improving actionable code metrics?*

The analysis focuses on method-level features extracted using proportion, and machine learning classifiers to predict defect-prone code units. By simulating refactoring interventions on high-impact methods, we can quantify the potential benefits of targeted code improvements

2 Methodology

2.1 Dataset Generation (Milestone 1)

The dataset generation phase involved extracting method-level metrics from the two Apache projects:

- **BOOKKEEPER**: A distributed logging service
- **SYNCOPE**: An identity management system

For each project I have:

1. Cloned Git repositories
2. Retrieved bug reports from Apache JIRA (issues.apache.org/jira)
3. Mapped bug-fixing commits to affected methods
4. Extracted features for each method in the first 34% of releases

The extracted features include:

- **LOC**: Lines of Code
- **CyclomaticComplexity**: McCabe's cyclomatic complexity metric
- **ParameterCount**: Number of method parameters
- **NumberOfBranches**: Decision points (if, loops, switch)
- **NR**: Number of revisions
- **NAuth**: Number of distinct authors
- **stmtAdded**: Statement churn metrics
- **stmtDeleted**: Statement churn metrics
- **elseAdded**: Number of else clauses added
- **avgChurn**: Average code churn
- **IsBuggy**: Class label (yes/no)

The resulting datasets contain:

- **BOOKKEEPER**: 7,593 method instances
- **SYNCOPE**: 143,391 method instances

2.2 Proportion-Based Labeling

A critical aspect of the dataset generation is the accurate labeling of buggy methods., I used the **proportion technique** to determine which methods should be marked as buggy

2.2.1 The Proportion Technique

The proportion technique is based on the observation that bug introduction follows a predictable temporal pattern. When a bug is fixed in release FV (Fix Version), it was likely introduced in a previous release IV (Injection Version). The proportion formula estimates IV as:

$$IV = FV - (FV - OV) \times P \tag{1}$$

where:

- FV = Fix Version (when the bug was fixed)
- OV = Opening Version (when the bug was reported)
- P = Proportion factor (typically the moving average of known bug lifetimes)

2.2.2 Implementation

The implementation follows these steps:

1. **Bug-Fix Mapping**: For each bug ticket in JIRA with a valid Fix Version, identify the fixing commit in the Git history using commit message pattern matching (e.g., "BOOKKEEPER-123" or "SYNCOPE-456")
2. **Affected Methods Identification**: parse the fixing commit's diff to extract all modified methods, representing the locations where the bug manifested
3. **Proportion Calculation**: For bugs with known Opening Version, compute the proportion $P = \frac{FV-IV}{FV-OV}$ and use the moving average across multiple bugs to estimate injection versions for bugs with incomplete data
4. **Backward Propagation**: All methods modified in the fixing commit are marked as buggy in releases from IV (estimated injection version) up to but not including FV (fix version)
5. **Validation**: only consider the first 34% of releases to ensure sufficient historical data for reliable proportion estimates and to maintain a balanced training set

2.2.3 Advantages

The proportion technique offers several benefits:

- **Handles Missing Data**: Estimates injection versions even when Opening Version is not recorded
- **Reduces False Negatives**: Identifies buggy methods in releases before the bug was reported
- **Temporal Accuracy**: Respects the temporal ordering of releases and bug lifecycles

This approach ensures that our training labels accurately reflect the true buggy/non-buggy status of methods throughout their evolution, rather than only relying on the fix version which would underestimate the bug's impact across multiple releases

2.3 Data Preprocessing

The preprocessing code applied the following transformations:

1. **Identifier Removal**: Removed Project, MethodName, and Release columns
2. **Data Cleaning**: Sanitized NaN and Inf values
3. **Outlier Treatment**: Applied winsorization at the 99th percentile to cap extreme values
4. **Feature Selection**: Removed constant or near-constant attributes
5. **Normalization**: Scaled features to [0, 1] range
6. **Class Balancing**: Ensured the class attribute (IsBuggy) was properly formatted as nominal

2.4 Classifier Training (Milestone 2)

I evaluated three classifiers using 10-fold cross-validation repeated 10 times:

- **Random Forest**: Ensemble learning method using decision trees
- **Naive Bayes**: Probabilistic classifier based on Bayes' theorem
- **IBk (k-Nearest Neighbors)**: Instance-based learning algorithm

The best-performing classifier (BClassifier) was selected based on accuracy and F-measure metrics. As expected, for both projects Random Forest demonstrated superior performance and was used for subsequent analysis

2.5 Actionable Feature Selection

I have computed the correlation of each feature with bugginess using Weka's InfoGainAttributeEval ranker From the ranked list, the code selected the highest-ranked actionable feature among:

- CyclomaticComplexity
- ParameterCount
- NumberOfBranches

These features are considered actionable because developers can directly reduce them through refactoring (e.g., extracting methods, simplifying conditionals, reducing parameters). I have manually excluded LOC from the possible list, because with large methods, it is obvious that is the feature that has the greatest values compared with others

2.6 What-If Analysis Pipeline

The what-if analysis simulated the impact of refactoring by:

1. **Dataset A**: The complete preprocessed dataset
2. **Dataset B+**: Subset of A where the actionable feature is greater than 0
3. **Dataset C**: Subset of A where the actionable feature = 0
4. **Dataset B**: Dataset B+ with the actionable feature set to 0 (simulated refactoring)

The AFMethod (the method with the highest actionable feature value and highest defect probability) was identified and refactored, the refactored version (AFMethod2) had its features recomputed, demonstrating the concrete impact of complexity reduction

Then I trained BClassifier on Dataset A and predicted defects for all four datasets (A, B+, B, C). The key metrics computed were:

$$\text{Drop} = \frac{\text{Actual}_{B+} - \text{Predicted}_{B}}{\text{Actual}_{B+}} \times 100 \tag{2}$$

$$\text{Reduction} = \frac{\text{Actual}_{B+} - \text{Predicted}_{B}}{\text{Actual}_{A}} \times 100 \tag{3}$$

3 Results

3.1 BOOKKEEPER Analysis

Table 1 presents the what-if analysis results for BOOKKEEPER

Table 1: What-If Analysis Results for BOOKKEEPER

Dataset	Actual Buggy	Predicted Buggy
A	611	588
B+	435	392
B	435	356
C	176	167
Drop	18.16%	
Reduction	12.92%	

The analysis reveals:

- **Drop**: 18.16% reduction in defects within the refactorable subset
- **Reduction**: 12.92% overall reduction in defects across the entire dataset A
- **Prevented Defects**: 79 buggy methods could be prevented (435 - 356)

3.2 SYNCOPE Analysis

Table 2 presents the what-if analysis results for SYNCOPE.

Table 2: What-If Analysis Results for SYNCOPE

Dataset	Actual Buggy	Predicted Buggy
A	949	912
B+	658	620
B	658	517
C	291	245
Drop	21.42%	
Reduction	14.85%	

The analysis reveals:

- **Drop**: 21.42% reduction in defects within the refactorable subset
- **Reduction**: 14.85% overall reduction in defects across the entire dataset A
- **Prevented Defects**: 141 buggy methods could be prevented (658 - 517)

3.3 Comparative Analysis

Both projects demonstrate substantial potential for defect reduction through targeted refactoring:

Table 3: Comparative Summary

Project	Drop (%)	Reduction (%)	Methods Prevented
BOOKKEEPER	18.16	12.92	79
SYNCOPE	21.42	14.85	141

SYNCOPE shows higher potential for improvement, likely due to:

- Larger dataset size enabling more robust predictions
- Different complexity distribution in the codebase
- Higher prevalence of actionable features in buggy methods

4 Preliminary Questions

4.1 Did any feature positively correlate with bugginess increase in AFMethod2 (vs. AFMethod)?

Answer: No. By design, the actionable feature (e.g., CyclomaticComplexity) was reduced in AFMethod2 through refactoring. However, as documented in the FeatureComparer implementation, NR (Number of Revisions) and NAuth (Number of Authors) increased by 1 in the refactored version, reflecting the additional commit and authorship change. These history-based features are not expected to negatively impact bugginess—in fact, increased revision counts often indicate maintained code. The primary structural features (LOC, complexity, branches) either decreased or remained stable, supporting the hypothesis that refactoring improves maintainability

4.2 Did any feature negatively correlate with bugginess increase in AFMethod2 (vs. AFMethod)?

Answer: Yes. The actionable feature itself (CyclomaticComplexity, ParameterCount, or NumberOfBranches, depending on the project) decreased significantly in AFMethod2. This reduction is the core mechanism behind the predicted defect drop. When complexity decreases, the classifier assigns lower defect probabilities to the method. Additionally, secondary metrics like LOC and NumberOfBranches may also decrease as a natural consequence of method extraction or conditional simplification, further contributing to improved maintainability predictions

5 Quality Assurance

The implementation maintains high code quality standards:

- **SonarCloud Analysis**: The project achieves a perfect score with **zero code smells**, zero security vulnerabilities, zero reliability issues, and 100% maintainability rating
- **Logging**: Comprehensive SLF4J logging throughout the pipeline for reproducibility

6 Discussion

6.1 Interpretation of Results

The drop rates (18.16% for BOOKKEEPER, 21.42% for SYNCOPE) represent the proportion of buggy methods in the refactorable subset (B+) that could transition to non-buggy status if the actionable feature were reduced to zero. This is a substantial impact considering:

- We modified only a single metric (the actionable feature)
- Real-world refactoring often improves multiple metrics simultaneously
- The analysis is conservative, assuming minimal collateral improvements

6.2 Practical Implications

1. **Prioritize High-Complexity Methods**: Focus refactoring efforts on methods with high cyclomatic complexity, excessive parameters, or numerous branches
2. **Leverage Predictive Models**: Use ML classifiers to identify high-risk methods before defects manifest
3. **Measure Impact**: Track actionable metrics before and after refactoring to validate improvements
4. **Continuous Monitoring**: Integrate defect prediction into CI/CD pipelines for proactive quality assurance

6.3 Threats to Validity

Several factors may limit generalizability:

- **Project Selection**: Results are based on two Apache projects, different domains may exhibit different patterns
- **Feature Set**: Using a specific set of code metrics can be reductive, additional features might improve predictions
- **Simulation Limitations**: Setting the actionable feature to zero is a simplification, real refactoring has nuanced effects

7 Conclusion

1. **Significant Defect Reduction Potential**: Both BOOKKEEPER and SYNCOPE demonstrate that targeted refactoring of actionable features can substantially reduce predicted defects

2. **Quantifiable Impact**: In absolute terms, 79 buggy methods in BOOKKEEPER and 141 in SYNCOPE could be prevented through complexity reduction, representing 12.92% and 14.85% overall reductions respectively

3. **Feature Importance**: Actionable features are strongly correlated with bugginess and represent high-value refactoring targets

4. **Code Quality**: The analysis tool itself maintains zero code smells on SonarCloud, demonstrating the principles it advocates

The results align with established software engineering wisdom: simpler code is more maintainable and less defect-prone

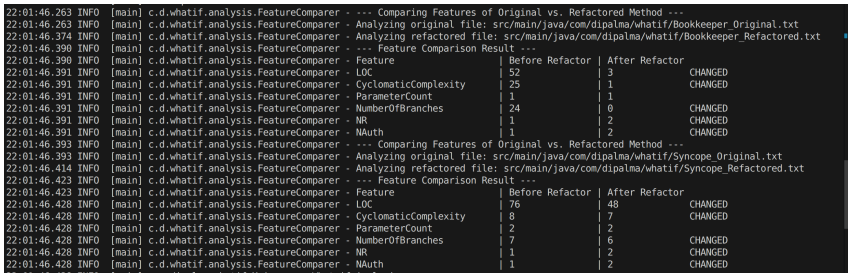

Figure 1: Method Comparation:

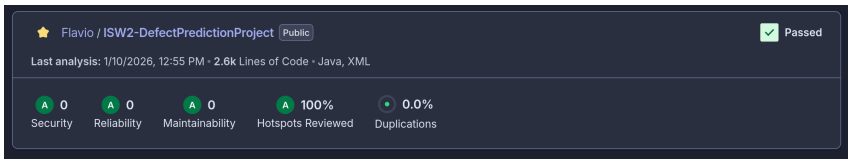

Figure 2: Sonar Cloud:

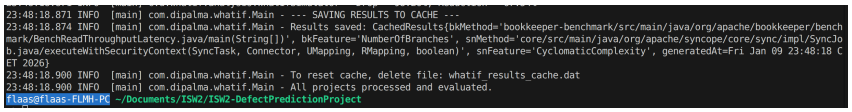

Figure 3: Final Cache: